

数据结构与算法应用

固定的下午考试题，分数15分，必答题

主要分数集中在

- 常见数据结构和算法的C语言实现。

1、试题1：常见数据结构和算法

【说明】

某工程计算中经常要完成多个矩阵相乘(链乘)的计算任务，对矩阵相乘进行以下说明。

(1)两个矩阵相乘要求第一个矩阵的列数等于第二个矩阵的行数，计算量主要由进行乘法运算的次数决定，假设采用标准的矩阵相乘算法，计算 $A_{m \times n} \times B_{n \times p}$ 需要 $m \times n \times p$ 次行乘法运算的次数决定、乘法运算，即时间复杂度为 $O(m \times n \times p)$ 。

(2)矩阵相乘满足结合律，多个矩阵相乘时不同的计算顺序会产生不同的计算量。以矩阵 $A_{15 \times 100}$ ， $A_{100 \times 8}$ ， $A_{8 \times 50}$ 三个矩阵相乘为例，若按 $(A_1 \times A_2) \times A_3$ 计算，则需要进行 $5 \times 100 \times 8 + 5 \times 8 \times 50 = 6000$ 次乘法运算，若按 $A_1 \times (A_2 \times A_3)$ 计算，则需要进行

$100 \times 8 \times 50 + 5 \times 100 \times 50 = 65000$ 次乘法运算。

矩阵链乘问题可描述为:给定 n 个矩阵，对较大的 n ,可能的计算顺序数量非常庞大，用蛮力法确定计算顺序是不实际的。经过对问题进行分析，发现矩阵链乘问题具有最优子结构，即若 $A_1 \times A_2 \times \dots \times A_n$ 的一个最优计算顺序从第 k 个矩阵处断开，即分为 $A_1 \times A_2 \times \dots \times A_k$ 和 $A_{k+1} \times A_{k+2} \times \dots \times A_n$ 两个子问题，则该最优解应该包含 $A_1 \times A_2 \times \dots \times A_k$ 的一个最优计算顺序和 $A_{k+1} \times A_{k+2} \times \dots \times A_n$ 的一个最优计算顺序。据此构造递归式，

$$\text{cost}[i][j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \text{cost}[i][k] + \text{cost}[k+1][j] + p_i * p_{k+1} * p_{j+1} & \text{if } i < j \end{cases}$$

其中， $\text{cost}[i][j]$ 表示 $A_{i+1} \times \dots \times A_{j+1}$ 的最优计算的计算代价。最终需要求解 $\text{cost}[0][n-1]$ 。

【C代码】

算法实现采用自底向上的计算过程。首先计算两个矩阵相乘的计算量，然后依次计算3个矩阵、4个矩阵、... n 个矩阵相乘的最小计算量及最优计算顺序。下面是该算法的语言实现。

(1) 主要变量说明

n: 矩阵数

seq[]: 矩阵维数序列

cost[i][j]: 二维数组, 长度为n*n, 其中元素cost[i][j]表示 $A_{i+1} \cdot A_{i+2} \cdot \dots \cdot A_{j+1}$ 的最优的计算代价。

trace[i][j]: 二维数组, 长度为n*n, 其中元素trace[i][j]表示 $A_{i+1} \cdot A_{i+2} \cdot \dots \cdot A_{j+1}$ 的最优计算顺序对应的划分位置, 即k。

(2) 函数cmm

```
#define N100
int cost[N][N];
int trace[N][N];
int cmm(int n,int seq[]){
    int tempCost;
    int tempTrace;
    int i,j,k,p;
    int temp;
    for( i=0;i<n;i++){ cost[i][i] = 0;}
    for(p=1;p<n;p++){
        for(i=0; i<n-p;i++){
            (1) ;
            tempCost = -1;
            for(k = i; (2) ;k++){
                temp= (3) ;
                if(tempCost== -1 || tempCost>temp){
                    tempCost = temp;
                    tempTrace=k;
                }
            }
            cost[i][i+p] = tempCost;
            (4) ;
        }
    }
    return cost[0][n-1];
}
```

【问题1】 (8分)

根据以上说明和C代码, 填充C代码中的空(1) ~ (4)。

【问题2】 (4分)

根据以上说明和C代码, 该问题采用了(5) 算法设计策略,时间复杂度为(6) (用O符号表示)。

【问题3】 (3分)

考虑实例n=4, 各个矩阵的维数为A1为15*5, A2为5*10, A3为10*20, A4为20*25, 即维度序列为15, 5, 10, 20和25。 则根据上述C代码得到的一个最优计算顺序为(7) (用加括号 方式表示计算顺序), 所需要的乘法运算次数为(8)。

【参考答案】

[问题1]

- (1) $j=i+p$
- (2) $k<j$
- (3) $cost[i][k]+cost[k+1][j]+seq[i]*seq[k+1]*seq[j+1]$
- (4) $trace[i][j]=tempTrace$

[问题2]

(5) 动态规划算法

(6) $O(n^3)$

[问题3]

(7) $A1*((A2*A3)*A4)$

(8) 5375

2、试题2

阅读下列说明和C代码，回答问题1至问题3，将解答写在答题纸的对应栏内。[说明]

排序是将一组无序的数据元素调整为非递减顺序的数据序列的过程，堆排序是一种常用的排序算法。用顺序存储结构存储堆中元素。非递减堆排序的步骤是：

1)将含n个元素的待排序数列构成一个初始大顶堆，存储在数组R[F1], R12, 中。此时堆的规模为 , 堆顶元素R[1]就是序列中最大的元素，R[n]是堆中最后一个元素。

2)将堆顶元素和堆中最后一个元素交换，最后一个元素脱离堆结构，堆的规模减1，将堆中剩余的元素调整成大顶堆:(3)重复步骤(2)，直到只剩下最后一个元素在堆结构中，此时数组R是一个非递减的数据序列。

【C代码】

下面是该算法的C语言实现。

(1)主要变量说明

n:待排序的数组长度

R[]: 待排序数组, n个数放在R[1], R[2], ..., R[n]中

(2) 代码

```
#include <stdio.h>
```

```
#define MAXITEM 100
```

```
/*
```

调整堆

R:待排序数组:

v:节点编号, 以v为根的二叉树, $R[v] \geq R[2v]$, $R[v] \geq R[2v+1]$, 且其左子树和右子树都是大顶堆:

n:堆结构的规模, 即堆中的元素数

```
*/
```

```
void Heapify(int R[MAXITEM],int v,int n){
```

```
int i,j;
```

```
i=v;
```

```
j=2*i;
```

```
R[0]=R[i];
```

```
while (j<=n){
```

```
    if(j<n&& R[j]<R[j+1]){
```

```
        j++;
```

```
    }
```

```

if(____(1)____){
    R[i]=R[j];
    i=j;
    j=2*i;
}
else{
    j=n+1;
}
}
R[i]=R[0];
}
/* 堆排序, R为待排序数组; n为数组大小 */
void HeapSort(int R[MAXITEM],int n){
    int i;
    for(i=n/2; i>=1;i--){
        (____(2)____);
    }
    for(i=n;(____(3)____);i--){
        R[0]=R[i];
        R[i]=R[1];
        ____ (4) ____;
        Heapify (R,1,i-1);
    }
}

```

[问题1] (8分)

根据以上说明和C代码, 填充C代码中的空(1)~(4)。

[问题2] (2分)

根据以上说明和C代码, 算法的时间复杂度为(5) (用O符号表示)。

[问题3] (5分)

考虑数据序列R=(7, 10, 13, 15, 4, 20, 19, 8), n=8, 则构建的初始大顶堆为(6), 第一个元素脱离堆结构, 对剩余元素再调整成大顶堆后的数组R为 (7) 。

【参考答案】

[问题1]

1: $R[0] < R[j]$

2: Heapify(R,i,n)

3: $i > 1$

4: $R[1]=R(0)$

[问题2]

5: $O(n \log n)$ 或 $O(n \lg n)$

[问题3]

6: {20, 15, 19, 10, 4, 13, 7, 8}或其树形结构。

7: {19, 15, 13, 10, 4, 8, 7, 20}